```
<div style="float: right;">
  {% if user.is_authenticated %}
    Hello, {{user.username}}!  

    <form id="logout-form" method="post"
        action="{% url 'logout' %}"
        style="display: inline;">
      {% csrf_token %}
      <button type="submit"
        style="background: none; border: none; cursor: pointer;
        ➥ padding: 0; text-decoration: underline;"
        >Logout</button>
    </form>
  {% else %}
    <a href="{% url 'login' %}">Login</a>
  {% endif %}
</div>

<!-- ... -->
```

Style the nav so that it pulls to the right of the page.

If the user is authenticated, greet them.

The <form> tag needed to log a user out

A link to the login page

The `<form>` for logging out a user is a bit messy and includes some horrific inline CSS styling. Forms don't work with links; they require buttons. And buttons don't look like links; they look like buttons. That long, ugly style chunk is to make the button look more like a link.

You can configure Django to go where you want when a user logs out. A common choice is to send them to the home page. As you haven't built a home page yet, let's send them to the band listing page. Add the following snippet to your settings file:

```
# RiffMates/RiffMates/settings.py
...
# Account Management
LOGIN_REDIRECT_URL = '/bands/bands/'
LOGOUT_REDIRECT_URL = '/bands/bands/'
```

Where to redirect to if there is no next argument provided to the login page

Where to redirect to when the user logs out

6.5 Using signals to automatically create a user profile

The `musician_restricted` view has a dangerous assumption in it: it directly accesses the user's associated user profile object without validating it exists. If you log in as the admin account and visit the view, you see a `500` error `RelatedObjectDoesNotExist`, as the admin doesn't have a user profile.

At the moment, the user profile object only gets created if you create a `User` in the Django Admin and that user has either a `Musician` or `Venue` associated with it. There are two solutions to this problem; the simplest is to always check for the existence of a user profile before using it. In the previous example, you could raise a `404` error if the user profile does not exist.

Alternatively, you can write code that automatically creates a `UserProfile` object immediately after a `User` ORM object is made. This requires a bit more programming, but it makes your code safer, as forgetting to check for a user profile's existence won't crash your site.